LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
SEMESTER III



TUGAS PRAKTIKUM ACARA 26

NAMA:

RAJATA PRINGGIT BINTANG BIHAR

GOLONGAN:

A

NIM:

E41250264

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Pada Acara 25, dashboard admin sudah dilindungi autentikasi dan pembatasan peran, tetapi data mahasiswa baru berupa teks. Acara 26 menambahkan fitur manajemen konten berupa foto mahasiswa yang diunggah lewat form dashboard dan disimpan di folder *public/uploads*, serta latihan manajemen basis data berupa pencadangan (*backup*) dan pemulihan (*restore*) *db_workshop* melalui phpMyAdmin.

2. Tujuan

1. Menambahkan kolom foto pada tabel mahasiswa dan menyiapkan folder penyimpanan berkas.
2. Memperbarui Model dan MahasiswaController agar menerima dan memvalidasi unggahan foto.
3. Memperbarui form dashboard agar mengirim berkas dengan FormData.
4. Melakukan backup dan restore basis data melalui phpMyAdmin dan memverifikasi hasilnya.
5. Menghapus foto lama dengan unlink() saat foto diganti (tugas mandiri 1).
6. Menjadwalkan dan mencatat backup basis data setiap akhir sesi praktikum (tugas mandiri 2).

3. Dasar Teori

PHP menampung informasi berkas yang diunggah (nama, ukuran, tipe, lokasi sementara) pada variabel superglobal *$_FILES*, lalu *move_uploaded_file()* memindahkan berkas dari direktori sementara ke tujuan permanen. Fitur ini berisiko karena penyerang dapat mengunggah skrip berbahaya. Validasi tidak boleh bertumpu pada ekstensi atau *$_FILES['type']* yang dapat dipalsukan; *exif_imagetype()* memeriksa signature berkas yang sesungguhnya. Praktik lain: membatasi ukuran dan mengganti nama berkas dengan pengenal unik seperti *uniqid()* agar tidak saling menimpa.
Backup adalah pencadangan berkala untuk mengantisipasi kehilangan data. phpMyAdmin menyediakan *Export* untuk menghasilkan berkas SQL dan *Import* untuk memulihkannya. Cadangan baru berguna bila terbukti dapat dipulihkan, sehingga hasil restore perlu diverifikasi.

HASIL PRAKTIK

1. Implementasi

a. Perubahan File

Acara 26 mengubah Model, Controller, dan Router di sisi server, menambahkan folder uploads, dan memperbarui tampilan dashboard. Dokumentasi, koleksi Postman, serta skrip backup ikut diperbarui.

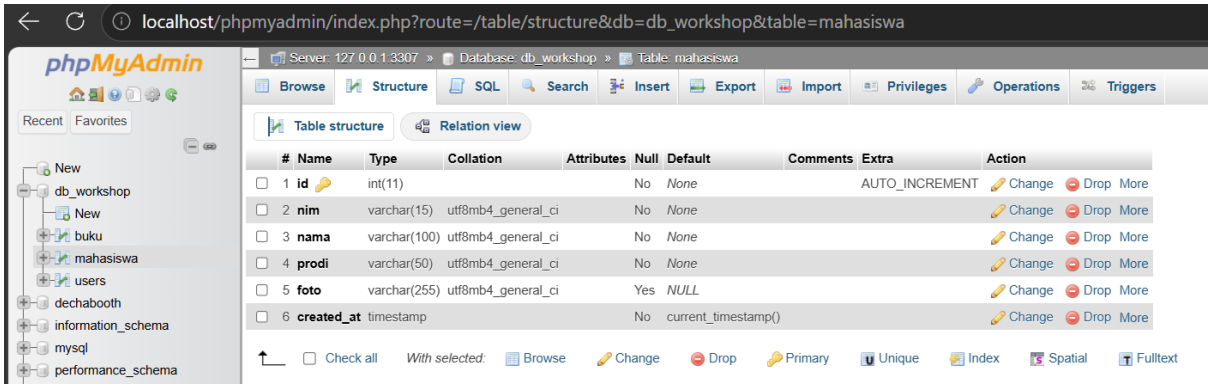
Tabel 1. Daftar perubahan file

File	Status	Keterangan
app/models/Mahasiswa.php	Diubah	create() menyimpan foto; update() mengubah foto bila ada foto baru
app/controllers/MahasiswaController.php	Diubah	Validasi dan penyimpanan foto, unlink() foto lama
app/core/Router.php	Diubah	POST dengan _method=PUT diperlakukan sebagai PUT
public/uploads/ (+ .htaccess)	Baru	Folder foto; eksekusi skrip dilarang
db_acara26.sql	Baru	ALTER TABLE kolom foto
public/views/common.js, dashboard.html, dashboard.js, style.css	Diubah	FormData, kolom Foto, pratinjau, validasi awal
backup/backup_db.bat, README_BACKUP.md	Baru	Backup berjadwal dan panduan restore
docs/*, postman/ACARA26_TestCase...json	Diubah	Dokumentasi upload; regresi 29 test case

b. Langkah 1 - Menambahkan Kolom foto pada Tabel mahasiswa

Kolom foto bertipe VARCHAR(255) dan boleh kosong (NULL) karena foto bersifat opsional dan data lama tidak memilikinya. Kolom hanya menyimpan nama berkas, bukan isi gambar.

```
ALTER TABLE mahasiswa ADD COLUMN foto VARCHAR(255) DEFAULT NULL AFTER prodi;
```



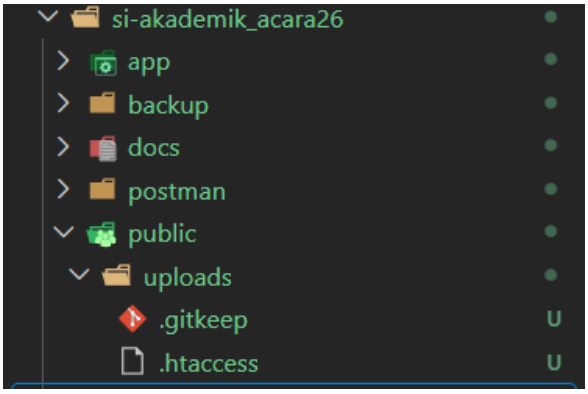
Gambar 1. Struktur tabel mahasiswa pada phpMyAdmin dengan kolom foto

c. Langkah 2 - Membuat Folder Penyimpanan Berkas

Folder uploads berada di dalam public/ agar berkas dapat dibuka langsung oleh browser. Karena folder ini menerima berkas dari pengguna, ditambahkan .htaccess yang melarang eksekusi skrip dan menutup daftar isi folder, sebagai lapisan pengaman kedua bila ada berkas berbahaya yang lolos validasi.

```
public/
+-- uploads/
|   +-- .htaccess
|   +-- (berkas foto tersimpan di sini)
+-- views/
+-- index.php
```

```
# public/uploads/.htaccess
Options -Indexes
<IfModule php_module>
    php_flag engine off
</IfModule>
<FilesMatch "\.(php|phtml|phar|php[0-9])$">
    Require all denied
</FilesMatch>
```



Gambar 2. Struktur folder public/uploads

d. Langkah 3 - Memperbarui Model Mahasiswa

create() kini menyimpan kolom foto, bernilai NULL bila tidak ada foto. update() hanya menyentuh kolom foto bila ada foto baru, sehingga mengubah data teks tidak menghapus foto yang sudah ada.

```
// app/models/Mahasiswa.php (cuplikan)
        $stmt = $this->db->prepare(
                'INSERT INTO mahasiswa (nim, nama, prodi, foto) VALUES (:nim,
:nama, :prodi, :foto)'
            );
            $stmt->execute([':nim' => $data['nim'], ':nama' => $data['nama'],
```

```
                ':prodi' => $data['prodi'], ':foto' => $data['foto'] ??
null]);

// update(): kolom foto diubah hanya bila ada foto baru
            $set = 'nim = :nim, nama = :nama, prodi = :prodi';
            if (!empty($data['foto'])) {
                $set .= ', foto = :foto';
                $params[':foto'] = $data['foto'];
            }
```

e. Langkah 4 - Menambahkan Logika Upload pada MahasiswaController

Logika upload ditulis sebagai method prosesFoto() yang dipanggil oleh store() dan update(). Dibanding cuplikan modul terdapat beberapa penyesuaian yang ditemukan saat pengujian, dirangkum pada Tabel 2 dan dibahas pada bagian Pembahasan.

```
 // MahasiswaController::prosesFoto() (cuplikan)
    $file = $_FILES['foto'] ?? null;
        if ($file === null || $file['error'] === UPLOAD_ERR_NO_FILE) {
            return null;
        }
        $pesanGagal = ['foto' => 'Foto tidak dapat diunggah'];
        if ($file['error'] === UPLOAD_ERR_INI_SIZE || $file['error'] ===
UPLOAD_ERR_FORM_SIZE
            || $file['size'] > 2 * 1024 * 1024) {
            Response::json(422, false, 'Ukuran berkas maksimal 2MB', null,
['foto' => 'Ukuran berkas maksimal 2MB']);
        }
        if ($file['error'] !== UPLOAD_ERR_OK) {
            Response::json(422, false, 'Berkas gagal diunggah, coba lagi',
null, $pesanGagal);
        }

        $tipe = @exif_imagetype($file['tmp_name']);
        $peta = [IMAGETYPE_JPEG => 'jpg', IMAGETYPE_PNG => 'png'];
        $ekstensi = strtolower(pathinfo($file['name'], PATHINFO_EXTENSION));
        if ($ekstensi === 'jpeg') $ekstensi = 'jpg';
        if (!isset($peta[$tipe]) || $peta[$tipe] !== $ekstensi) {
            Response::json(422, false, 'Berkas harus berupa gambar JPG/PNG
yang valid', null,
                ['foto' => 'Berkas harus berupa gambar JPG/PNG yang valid']);
        }

        $namaBaru = uniqid('foto_', true) . '.' . $peta[$tipe];
        if (!move_uploaded_file($file['tmp_name'], $this->folderUpload() .
$namaBaru)) {
            Response::json(500, false, 'Terjadi kesalahan pada server');
        }
```

```
        return $namaBaru;
    }
```

Tabel 2. Perbedaan dengan cuplikan modul

Hal	Cuplikan modul	Implementasi ini
Deteksi berkas	!empty($_FILES['foto'])	Memeriksa error UPLOAD_ERR_NO_FILE; form tanpa berkas tetap menghasilkan $_FILES['foto'] sehingga cuplikan modul akan memproses berkas kosong
Pemeriksaan tipe	Ekstensi JPG/PNG dan exif_imagetype() bernilai benar	Tipe hasil exif_imagetype() harus JPG atau PNG dan cocok dengan ekstensi; GIF berekstensi .png ditolak
Ekstensi berkas simpanan	Dari nama berkas pengguna	Dari tipe hasil deteksi
Pembaruan dengan berkas	Tidak dibahas	POST ?id=..&_method=PUT, karena PHP tidak mengisi $_FILES pada PUT
Data ditolak (NIM duplikat)	Tidak dibahas	Berkas yang sudah dipindah dihapus kembali
Data dihapus	Tidak dibahas	Berkas foto ikut dihapus

Pada store(), pemeriksaan token dan role (requireAdmin) dan validasi data teks dilakukan lebih dulu, sebelum berkas diproses, sehingga permintaan yang ditolak tidak meninggalkan berkas di server. Bila NIM ternyata duplikat, berkas yang sudah dipindahkan dihapus kembali.

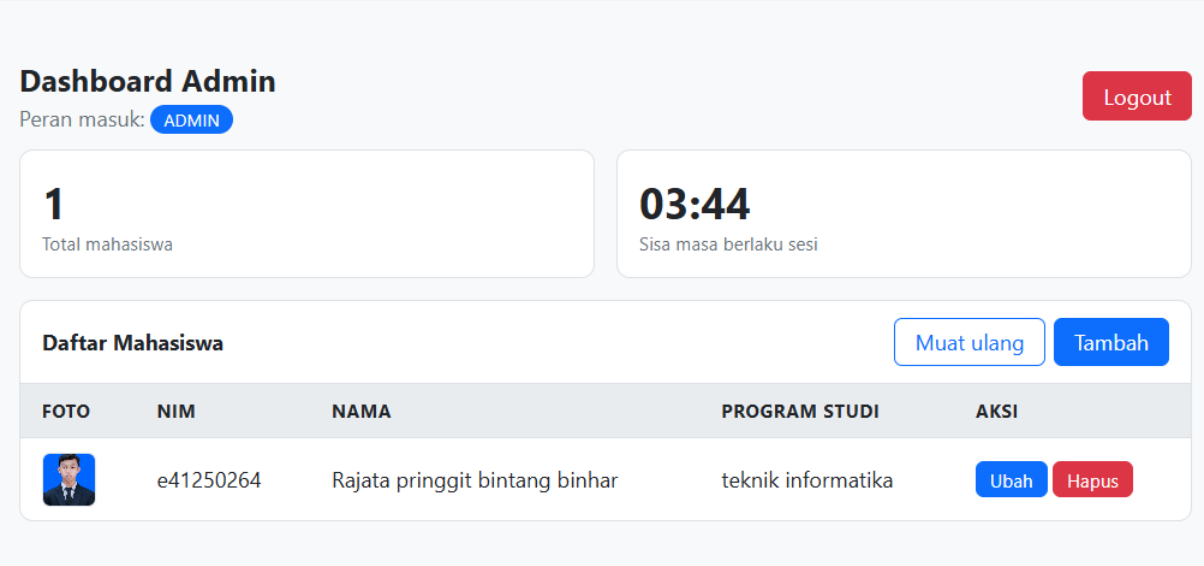
f. Langkah 5 - Memperbarui Form pada Dashboard Admin

Form tambah dan ubah kini memakai FormData karena request multipart tidak dapat dikirim sebagai JSON. Header Content-Type tidak diatur manual agar browser menambahkan boundary sendiri; fungsi panggilApi() mengenali FormData dan tidak mengubahnya menjadi JSON. Foto hanya dilampirkan bila dipilih, sebab append() dengan nilai undefined akan mengirim teks "undefined" sebagai nilai field. Pemeriksaan tipe dan ukuran juga dilakukan di browser untuk memberi umpan balik cepat; validasi yang menentukan tetap di server. Pembaruan dikirim sebagai POST dengan parameter _method=PUT.

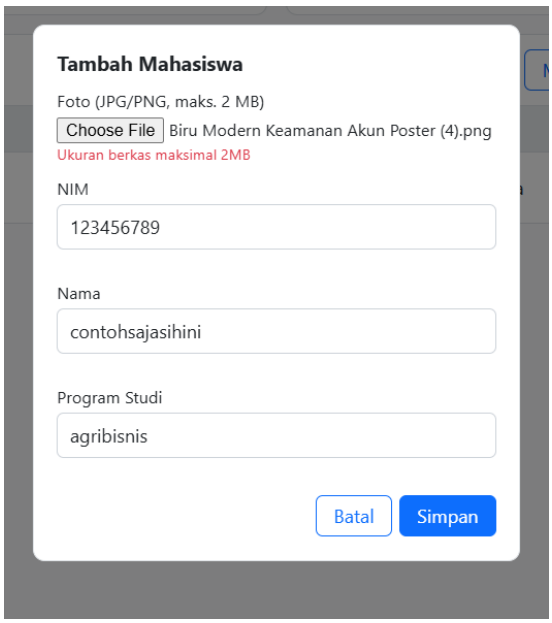
```
  // dashboard.js (cuplikan)
 const formData = new FormData();
        formData.append('nim', form.elements.nim.value.trim());
        formData.append('nama', form.elements.nama.value.trim());
        formData.append('prodi', form.elements.prodi.value.trim());
        if (file) formData.append('foto', file);
        const { ok, status, hasil } = await panggilApi(edit ?
`/mahasiswa?id=${idDiedit}&_method=PUT` : '/mahasiswa', {
            method: 'POST',
            auth: true,
            body: formData,
        });
```

```
// common.js: FormData tidak diberi Content-Type manual
   const multipart = body instanceof FormData;
   if (body !== null && !multipart) headers['Content-Type'] =
'application/json';
```

Tabel pada dashboard mendapat kolom Foto berisi gambar mini yang dibuat dengan createElement dan properti src, bukan innerHTML; mahasiswa tanpa foto menampilkan penanda "-". Form tambah/ubah menampilkan pratinjau foto dan pesan kesalahan pada field foto.



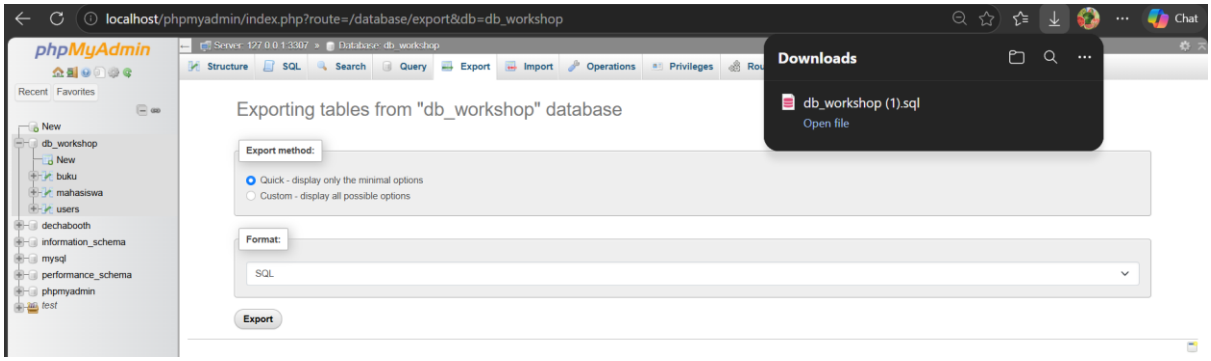
Gambar 3. Dashboard admin dengan kolom Foto



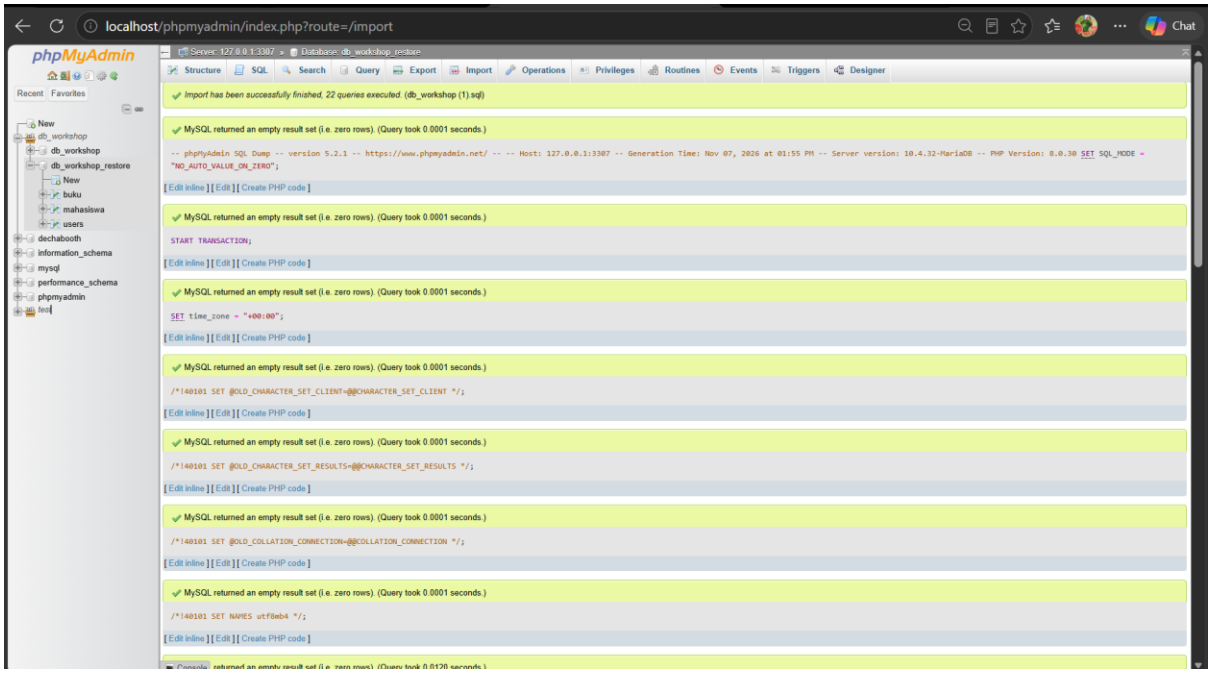
Gambar 4. Form dengan pesan kesalahan pada field foto

g. Langkah 6 - Backup dan Restore Basis Data

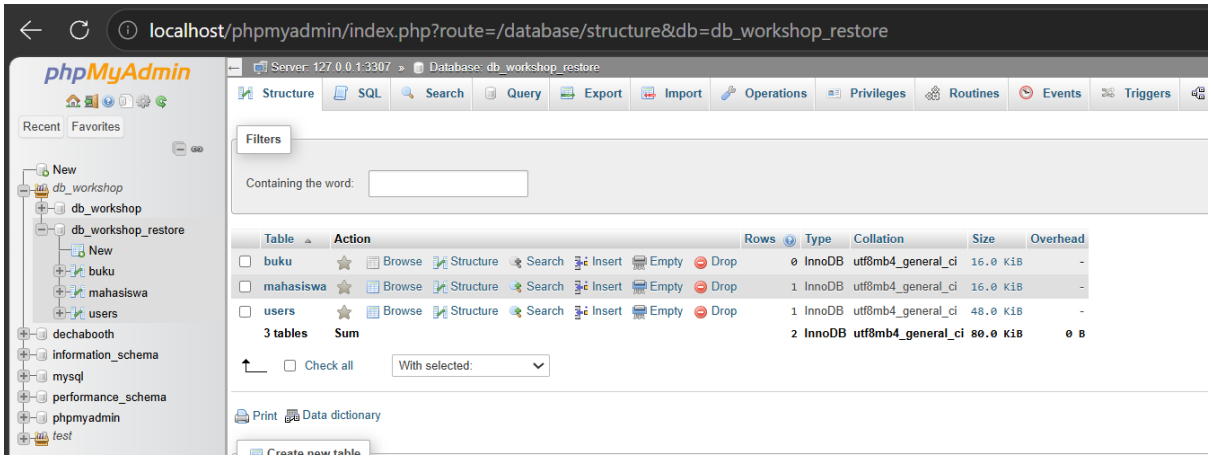
1. Pada phpMyAdmin pilih database db_workshop, buka tab Export, pilih metode Quick dan format SQL, lalu unduh berkas cadangan (Gambar 5).
2. Buat database percobaan db_workshop_restore, buka tab Import, pilih berkas cadangan, lalu jalankan (Gambar 6).
3. Verifikasi tabel users, mahasiswa, dan buku pada database percobaan dan bandingkan dengan database asal (Gambar 7).



Gambar 5. Export db_workshop pada phpMyAdmin



Gambar 6. Import ke database db_workshop_restore berhasil



Gambar 7. Tabel pada database hasil restore

Verifikasi tidak berhenti pada keberadaan tabel. Jumlah baris dan checksum isi setiap tabel pada database asal dibandingkan dengan database hasil restore, dan hasilnya sama (Tabel 3). Berkas foto tidak ikut dalam cadangan karena kolom foto hanya menyimpan nama berkas.

Tabel 3. Perbandingan database asal dan hasil restore

Pemeriksaan	**db_workshop**	**db_workshop_restore**	**Hasil**
Daftar tabel	users, mahasiswa, buku	users, mahasiswa, buku	Sama
Jumlah baris tiap tabel (COUNT)	users 1, mahasiswa 1, buku 0	users 1, mahasiswa 1, buku 0	Sama
Checksum tabel (CHECKSUM TABLE)	users 1509312430, mahasiswa 1318218276, buku 0	users 1509312430, mahasiswa 1318218276, buku 0	Sama
Mahasiswa yang memiliki foto	1	1	Sama

2. Tugas Praktik Mandiri

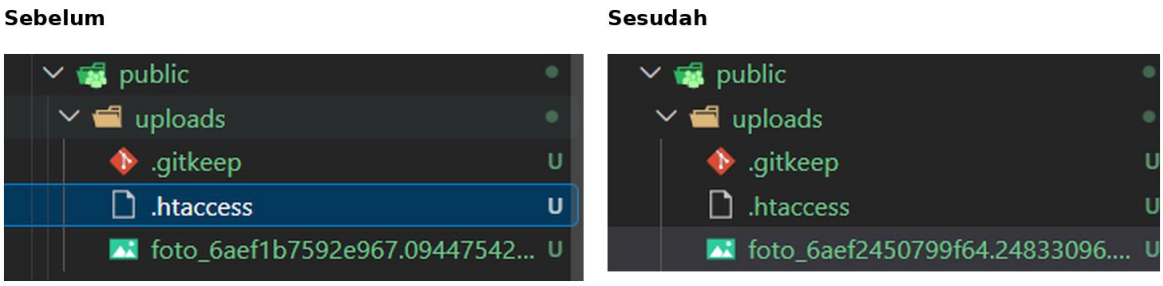
a. Menghapus foto lama dengan unlink()

Pada update(), setelah data berhasil diperbarui dengan foto baru, berkas foto lama dihapus. Penghapusan baru dilakukan setelah pembaruan basis data berhasil, sehingga bila pembaruan gagal (misalnya NIM duplikat) foto lama tetap utuh. Nama berkas dari basis data dilewatkan basename() agar tidak dapat dipakai menghapus berkas di luar folder uploads. Pada destroy(), berkas foto juga dihapus bersama datanya.

```
// MahasiswaController::update() (cuplikan)
$this->model->update((int) $id, $data);
      if ($fotoBaru !== null) $this->hapusBerkasFoto($lama['foto'] ?? null);
// unlink foto lama

    private function hapusBerkasFoto(?string $nama): void {
        if (!$nama) return;
        $path = $this->folderUpload() . basename($nama);
```

```
        if (is_file($path)) unlink($path);
    }
```



Gambar 8. Isi folder uploads sebelum dan sesudah foto diganti

b. Jadwal backup setiap akhir sesi praktikum

Backup dilakukan setiap akhir sesi praktikum (2 x 170 menit) agar kehilangan data paling banyak sebatas satu sesi. Script backup_db.bat membuat berkas SQL bertanggal dan menyalin folder uploads, dan dapat dijalankan otomatis dengan Task Scheduler Windows (Tabel 4).

Tabel 4. Jadwal backup basis data

Kegiatan	**Waktu**	**Cara**	**Keluaran**
Backup database dan foto	Akhir tiap sesi praktikum (pertemuan mingguan)	backup_db.bat atau Export phpMyAdmin	db_workshop_TTTTBBHH_JJMM.sql dan salinan folder uploads
Verifikasi cadangan	Awal sesi berikutnya, atau setelah perubahan struktur tabel	Import ke database percobaan, bandingkan jumlah baris	Catatan hasil verifikasi
Penyimpanan	Setelah backup	Salin ke penyimpanan lain (Git/Drive), simpan 4 cadangan terakhir	Cadangan di luar komputer praktikum

3. Pengujian

Pengujian dilakukan pada tiga lapisan terhadap server PHP dan MariaDB nyata. Fitur upload diuji dengan 31 skenario API memakai berkas uji yang sengaja dibuat bermasalah, dan setiap skenario memeriksa isi folder uploads. Regresi 29 test case Postman (25 test case Acara 23 dan 4 test case RBAC Acara 25) dijalankan ulang dengan Newman. Logika dashboard diuji dengan jsdom (38 skenario: 27 dari Acara 25 dan 11 skenario foto). Tampilan dan phpMyAdmin diperiksa langsung pada Gambar 1 sampai 8. Seluruh pemeriksaan sesuai harapan (Tabel 5).

Tabel 5. Hasil pengujian

No	Skenario	Hasil yang diharapkan	Hasil
1	Tambah dengan foto PNG dan JPG valid	201; berkas tersimpan dengan nama foto_*.png/jpg; foto tampil di browser (200, image/png)	Sesuai
2	Berkas yang sama diunggah dua kali	Dua nama berkas berbeda, tidak saling menimpa	Sesuai
3	Form tanpa berkas (bagian foto kosong) dan JSON tanpa foto	201; foto = null	Sesuai
4	Skrip PHP bernama .png, berkas .php	422 "Berkas harus berupa gambar JPG/PNG yang valid"; tidak ada berkas tersimpan	Sesuai
5	GIF berekstensi .png; PNG berekstensi .jpg	422; tidak ada berkas tersimpan	Sesuai
6	Gambar PNG 2,3 MB	422 "Ukuran berkas maksimal 2MB"	Sesuai
7	Token role user atau tanpa token mengunggah foto	403 / 401; tidak ada berkas tersimpan	Sesuai
8	NIM 3 digit atau field kosong, disertai foto valid	422 / 400; tidak ada berkas tersimpan	Sesuai
9	NIM duplikat disertai foto valid	409; berkas yang sudah dipindah dihapus kembali	Sesuai
10	PNG sah dengan kode PHP di ujung berkas	Diterima sebagai .png dan dilayani sebagai image/png, tidak dieksekusi	Sesuai
11	Ubah dengan foto baru (POST _method=PUT)	200; foto berganti; foto lama dihapus (unlink)	Sesuai
12	Ubah tanpa foto baru (multipart dan PUT JSON)	200; foto lama dan berkasnya dipertahankan	Sesuai
13	Ubah dengan berkas palsu; ubah ke NIM milik data lain	422 / 409; foto lama utuh; tidak ada berkas baru	Sesuai
14	Ubah dengan token user; ubah id tidak ada	403 / 404; tidak ada berkas tersimpan	Sesuai
15	POST dengan _method=DELETE	Tidak dianggap penghapusan (hanya PUT yang diakui); data tidak terhapus	Sesuai
16	Hapus data berfoto dan tanpa foto	200; baris dan berkas terhapus	Sesuai
17	Kolom foto berisi ../ pada basis data lalu data dihapus	Berkas di luar uploads tidak terhapus (basename)	Sesuai
18	Seluruh data dihapus	Folder uploads kosong, tidak ada berkas yatim	Sesuai
19	Dashboard: kolom Foto, thumbnail, pratinjau	Gambar mini dari /uploads tampil dan dapat dimuat; mahasiswa tanpa foto bertanda "-"	Sesuai
20	Dashboard: berkas bukan gambar atau 2,5 MB	Pesan pada field foto, tanpa permintaan ke server	Sesuai
21	Dashboard: berkas palsu bertipe image/png	Server 422, pesan tampil pada field foto, dialog tetap terbuka	Sesuai
22	Dashboard: tambah dan ubah dengan foto; ubah tanpa foto	Multipart tanpa Content-Type manual; foto berganti; foto lama dipertahankan bila tidak diganti	Sesuai
23	Dashboard role user	Foto tampil, tombol aksi tidak ada	Sesuai
24	Backup lalu restore ke database percobaan	Tabel, jumlah baris, dan checksum sama dengan database asal	Sesuai
25	Regresi 29 test case Postman (JSON, akun admin)	29 Pass, 0 Fail	Sesuai

Ringkasan: API upload 31 skenario Pass; dashboard 38 skenario Pass; Newman 31 request dan 35 assertion tanpa kegagalan.

4. Pembahasan

Keamanan upload dibangun berlapis. Ekstensi dan signature berkas harus sama-sama JPG/PNG dan cocok satu sama lain; ukuran dibatasi 2 MB; nama berkas dibuat ulang oleh server sehingga nama dari pengguna tidak pernah dipakai; ekstensi berkas simpanan berasal dari hasil deteksi, bukan dari nama kiriman; dan folder uploads dirancang melarang eksekusi skrip. Skrip PHP yang diberi nama .png ditolak karena signature-nya bukan gambar (skenario 4).

Pengujian menemukan beberapa hal yang tidak tercakup cuplikan modul. Pertama, form yang dikirim tanpa memilih berkas tetap membuat $_FILES['foto'] berisi error UPLOAD_ERR_NO_FILE, sehingga pengecekan !empty() saja akan memproses berkas kosong. Kedua, PHP tidak mengisi $_POST dan $_FILES untuk request PUT, sehingga pembaruan dengan foto tidak dapat memakai PUT multipart; solusinya POST dengan _method=PUT yang dibatasi hanya untuk PUT. Ketiga, tugas mandiri unlink() harus dijalankan setelah pembaruan basis data berhasil, bukan sebelumnya, agar foto lama tidak hilang bila pembaruan ditolak. Keempat, berkas yang sudah dipindahkan perlu dihapus kembali saat data ditolak karena NIM duplikat; tanpa itu folder uploads menyimpan berkas yatim.

Pemeriksaan exif_imagetype() hanya memastikan signature gambar. Pengujian skenario 10 menunjukkan PNG sah yang disisipi kode PHP di ujung berkas tetap diterima. Berkas itu aman karena disimpan sebagai .png, dilayani sebagai image/png, dan berada di folder yang melarang eksekusi skrip, tetapi pengamanan yang lebih ketat adalah membuat ulang gambar dengan GD sebelum disimpan. Backup basis data juga hanya memuat nama berkas pada kolom foto; tanpa salinan folder uploads, restore menghasilkan data yang merujuk foto yang tidak ada.

Keterbatasan: hanya format JPG dan PNG yang didukung (tanpa WebP/HEIC), satu foto per mahasiswa, gambar tidak diperkecil sehingga tabel memuat berkas ukuran penuh, dan pembatasan eksekusi pada .htaccess bergantung pada konfigurasi Apache yang mengizinkan override. Pengujian jsdom dan server uji tidak menggantikan Apache XAMPP dan browser sungguhan, karena itu tampilan dan phpMyAdmin diperiksa langsung pada Gambar 1 sampai 8. Backup dilakukan manual atau terjadwal dan tersimpan di komputer yang sama sampai disalin ke penyimpanan lain.

KESIMPULAN

1. Kolom foto, folder public/uploads, dan perubahan Model serta MahasiswaController berhasil dibuat; foto JPG/PNG maksimal 2 MB diterima, disimpan dengan nama unik, dan tampil pada dashboard.
2. Validasi berlapis (ekstensi, signature, ukuran, nama baru) menolak skrip dan berkas palsu tanpa meninggalkan berkas di server, dan berkas yang sudah dipindahkan dihapus kembali bila data ditolak.
3. Form dashboard mengirim berkas dengan FormData; pembaruan memakai POST dengan _method=PUT karena PHP tidak memproses multipart pada PUT.
4. Backup dan restore melalui phpMyAdmin berhasil dan terverifikasi: tabel, jumlah baris, dan isi database hasil restore sama dengan database asal, dengan catatan foto harus dicadangkan terpisah.
5. Tugas mandiri selesai: foto lama dihapus dengan unlink() setelah pembaruan berhasil, dan jadwal backup setiap akhir sesi praktikum dicatat beserta script-nya. Regresi 29 test case tetap Pass.

DAFTAR PUSTAKA

Pratama, R. A., & Hasanah, Q. (2026). *Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805)*. Program Studi D4 Teknik Informatika PSDKU Nganjuk, Jurusan Teknologi Informasi, Politeknik Negeri Jember.